

You're building a digital twin that **continuously understands what is happening across a property**. I spent two years at ElasticRun building the same class of system for warehouses — operations software whose model of physical reality had to stay correct through dead zones, concurrent operators and multiple enterprise tenants.

TWO YEARS AT ELASTICRUN, MEASURED

601 Jira items shipped over two years	4+ production apps — Field, Sales, Customer, Courier	474+ of those in Warehouse Management alone	1 yr+ production life of the framework I built as an intern
---	--	---	---

THREE THINGS I'VE ACTUALLY DONE

01 Keeping software honest about physical state Warehouse operators ran the Field App in steel racking and cold rooms where the network drops mid-operation. I worked the offline-first layer — GRN, putaway, picking, stock transfers — where a lost or double-applied scan means the client's stock ledger is simply wrong. Correctness there is not a UI concern.	02 Config-driven platforms, shipped without releases I proposed and built a campaigns delivery engine in ElasticRun's shared package layer: JSON config from a Frappe backend, parsed into typed objects so failures surface at the boundary, persisted locally, matched against in-app events at runtime, rendered by the package. Product shipped campaigns by changing config — no mobile release.	03 Spec-driven delivery, adopted by my team I write the contract before the code, then let AI assistants execute against it and review against it. Defects moved from post-release to pre-code. I promoted it by demonstration, not mandate — and it's the reason 601 items in two years is workflow rather than heroics.
---	--	--

YOUR REQUIREMENTS → MY EVIDENCE

Event streaming & real-time state	Built runtime event matching inside a live app, and an offline sync layer using client-generated operation IDs so a replayed message cannot double-apply — exactly-once over an unreliable channel.
Config-driven platform work	A delivery engine consuming JSON config from a Frappe backend — typed parsing, local persistence, self-rendering widgets. Adopted across ElasticRun's apps, live over a year.
Multi-tenant enterprise SaaS	Whitelabelled operator flows across multiple enterprise clients on shared infrastructure.
RAG, embeddings, vector search	Built retrieval with persistent memory and vector search into my own multi-agent system; currently building PermRAG, a permission-aware retrieval prototype with an evaluation harness.
Agent orchestration & evaluation	Designed agent topology, scoped context budgets, and explicit evaluation gates — and hit the real failure modes: context dilution, silent goal drift, plausible-but-irrelevant retrieval.
Python, REST APIs, Git	Two years shipping production software against REST backends across four applications.

BEING STRAIGHT WITH YOU

Two years full-time, roughly three professional. That's the lower edge of your band and I'd rather say it here than have you find it later.	What the number misses: 601 shipped items in two years is an unusually high rep count. I'd rather be tested on that than on my joining date.	What's mine, not my employer's: the multi-agent system and PermRAG are personal projects. I'll say so every time — but they're inspectable, which a résumé line isn't.
---	--	--

PROOF YOU CAN INSPECT

Offline-first sync · exactly-once	Config-driven campaigns engine	Multi-agent orchestration + eval gates	PermRAG · ACL-aware retrieval
B.Tech AI & DS · 9.3 CGPA · 2 papers			